🛡️ MDE Policy Automation

Guia Completo Passo-a-Passo com Referências Visuais

v1.0.4 Azure Policy PowerShell

Autor: Rafael França — Customer Success Architect, Cyber Security @ Microsoft
Projeto: github.com/rfranca777/MDE-PolicyAutomation
Comunidade: ODefender Community — Open-Source Security Automation

📋 Índice

1. Visão Geral da Solução
2. Pré-Requisitos
3. Etapa 1 — Autenticação e Seleção de Subscription
4. Etapa 2 — Nomenclatura Inteligente
5. Etapa 3 — Resource Group
6. Etapa 4 — Grupo Entra ID (Security Group)
7. Etapa 5 — Automation Account
8. Etapa 6 — Managed Identity (Zero Trust)
9. Etapa 7 — RBAC Reader
10. Etapa 8 — Permissões Graph API
11. Etapa 9 — Módulo Az.Accounts
12. Etapa 10 — Runbook PowerShell

13. Etapa 11 — Schedule e Job Schedule
14. Etapa 12 — Azure Policy (DeployIfNotExists)
15. Etapa 13 — MDE Device Groups
16. Etapa 14 — MDE Machine Tags via API
17. Validação Completa
18. Troubleshooting
19. Referências e URLs do Portal

1. Visão Geral da Solução

❌ O Problema

Em ambientes com **múltiplas subscriptions Azure**, as VMs Windows aparecem no portal MDE como uma lista única, sem organização:

🛡️ Microsoft Defender XDR — Device Inventory (SEM automação)

Device Name	OS	Tags	Device Group
srv-prod-01	Windows Server 2022	—	Ungrouped
srv-dev-03	Windows Server 2019	—	Ungrouped
vm-staging-02	Windows 11	—	Ungrouped
sql-prod-01	Windows Server 2022	—	Ungrouped
... centenas de VMs misturadas sem organização			

✅ A Solução

🛡️ Microsoft Defender XDR — Device Inventory (COM MDE Policy Automation)

Device Name	OS	Tags	Device Group
srv-prod-01	Windows Server 2022	🏷️ mde-policy-production	mde-policy-production
sql-prod-01	Windows Server 2022	🏷️ mde-policy-production	mde-policy-production
srv-dev-03	Windows Server 2019	🏷️ mde-policy-development	mde-policy-development
vm-staging-02	Windows 11	🏷️ mde-policy-staging	mde-policy-staging

🏗️ Arquitetura em 3 Camadas

🔵 CAMADA 1 — Azure Policy (Infraestrutura)

Azure Policy **DeployIfNotExists** detecta VMs Windows sem a extensão `MDEDeviceTagExtension` e automaticamente deploya o script `Set-MDEDeviceTag.ps1` que configura o registro Windows com o tag da subscription.

```
HKLM:\SOFTWARE\Policies\Microsoft\
  Windows Advanced Threat Protection\DeviceTagging
    Group = "mde-policy-{subscription-name}"
```

🟡 CAMADA 2 — Azure Automation (Operacional)

Automation Account com **Managed Identity** executa um Runbook PowerShell a cada hora que:

- Descobre todas as VMs Azure + Azure Arc machines na subscription
- Busca os devices correspondentes no Entra ID via Graph API
- Adiciona os devices encontrados ao Security Group do Entra ID

🔴 CAMADA 3 — MDE Integration (Segurança)

MDE Device Groups são configurados para filtrar devices pela tag. Cada subscription tem um Device Group específico, permitindo políticas AV/ASR diferenciadas por ambiente (PROD, DEV, TEST).

⚡ As 14 Etapas do Pipeline

Stage	O Que Acontece	Recurso Azure
1	🔐 Autenticação e seleção de subscription	Azure CLI context
2	🏷️ Nomenclatura inteligente baseada na subscription	Convenção: `rg-mde-{sub}`
3	📦 Resource Group com 8 tags corporativas	`Microsoft.Resources/resourceGroups`
4	👥 Entra ID Security Group	Microsoft Graph API
5	⚙️ Automation Account provisioning	`Microsoft.Automation/automationAccounts`
6	🔑 Managed Identity (Zero Trust)	`SystemAssigned` identity
7	🛡️ RBAC Reader role assignment	Subscription-scoped Reader
8	📡 Graph API permissions	`Group.ReadWrite.All` + `Device.Read.All`
9	📚 Módulo Az.Accounts	PowerShell Gallery → Automation
10	📜 Runbook criação e publicação	VM ↔ Entra ID sync logic
11	⏰ Schedule + Job Schedule linking	Execução a cada hora
12	📋 Azure Policy para VM tagging	`DeployIfNotExists` policy

13	🖥️ MDE Device Group instructions	HTML guide auto-gerado
14	🏷️ MDE Machine Tags via API	App Registration + OAuth2

2. Pré-Requisitos

Software Necessário

Requisito	Como Verificar	Onde Baixar
Azure CLI 2.0+	`az version`	aka.ms/installazurecli
PowerShell 5.1+	`$PSVersionTable.PSVersion`	Incluso no Windows 10/11
Git	`git --version`	git-scm.com

Permissões Azure

Permissão	Onde É Usada	Como Verificar
Contributor	Criar RG, Automation Account, Policy	Subscription → IAM
Policy Contributor	Criar e atribuir Azure Policy	Subscription → IAM
User Access Administrator	Atribuir RBAC para Managed Identity	Subscription → IAM
Global Admin	Graph API app permissions	Entra Admin Center → Roles

PowerShell — Verificação de Pré-Requisitos

```
PS C:\> az version
{
  "azure-cli": "2.68.0",
  "azure-cli-core": "2.68.0"
}

PS C:\> $PSVersionTable.PSVersion
Major  Minor  Patch
-----  -----  -----
7      4      6

PS C:\> az account show --query "{user:user.name, tenant:tenantId, sub:name}" -o
table
User                                   Tenant
Subscription
-------------------------------------  ------------------------------------  -------
-----------------
admin@contoso.onmicrosoft.com           9217e954-8efe-4920-ae1b-700c507c9faa
Production-Subscription
```

3. Etapa 1 — Autenticação e Seleção de Subscription

O Que Acontece

O script verifica se o Azure CLI está autenticado e lista todas as subscriptions disponíveis para o usuário selecionar qual deseja configurar.

📸 Saída do Terminal

```
Deploy-MDE-Automation.ps1 — Stage 1/14

============================================================
  MICROSOFT DEFENDER FOR ENDPOINT
  Deployment Completo - 14 Stages - AUTOMACAO TOTAL
  v1.0.4 - Full Automation Edition
============================================================

[1/14] AUTENTICACAO E SUBSCRICAO
==========================================================

  [OK] Autenticado: admin@contoso.onmicrosoft.com

  Subscriptions disponiveis:
  [1] ME-MngEnvMCAP186458-rafaelluizf-1
  [2] Development-Subscription
  [3] Staging-Subscription

  Selecione (1-3): 1
  [OK] Subscription: ME-MngEnvMCAP186458-rafaelluizf-1
```

📸 No Portal Azure — Subscriptions

☁️ Azure Portal — Subscriptions

🏠 Home > Subscriptions

Name	Status	Subscription ID
ME-MngEnvMCAP186458-rafaelluizf-1	✅ Active	fbb41bf3-dc95-4c71-8e14-396d3ed38b91

💡 **URL direta:** portal.azure.com → Subscriptions

4. Etapa 2 — Nomenclatura Inteligente

O Que Acontece

O nome da subscription é **sanitizado e padronizado** para gerar nomes únicos e consistentes para todos os recursos Azure que serão criados.

Lógica de Geração

PowerShell — Lógica de Nomenclatura

```
# Subscription: "ME-MngEnvMCAP186458-rafaelluizf-1"

$subNameClean = $subscriptionName -replace '[^a-zA-Z0-9-]', '-'  # Remove caracteres especiais
                                  -replace '--+', '-'          # Remove hífens duplicados
                                  -replace '^-|-$', ''         # Remove hífens nas pontas

$subNameShort = $subNameClean.Substring(0, 40).ToLower()  # Trunca em 40 chars

# Resultado: "me-mngenvmcap186458-rafaelluizf-1"
```

Tabela de Recursos Gerados

Recurso	Padrão	Valor Gerado
Resource Group	`rg-mde-{sub}`	`rg-mde-me-mngenvmcap186458-rafaelluizf-1`
Automation Account	`aa-mde-{sub}`	`aa-mde-me-mngenvmcap186458-rafaelluizf-1`
Entra ID Group	`grp-mde-{sub}`	`grp-mde-me-mngenvmcap186458-rafaelluizf-1`
MDE Tag Value	`mde-policy-{sub}`	**`mde-policy-me-mngenvmcap186458-rafaelluizf-1`**
Runbook	`rb-mde-sync-{sub}`	`rb-mde-sync-me-mngenvmcap186458-rafaelluizf-1`
Schedule	`sch-mde-{sub}`	`sch-mde-me-mngenvmcap186458-rafaelluizf-1`
Azure Policy	`pol-mde-tag-{sub}`	`pol-mde-tag-me-mngenvmcap186458-rafaelluizf-1`

⚠️ **CRÍTICO:** O **TagValue** (`mde-policy-{sub}`) é a chave que conecta todo o pipeline. Ele é escrito no registro Windows pela Azure Policy, lido pelo MDE Agent, e usado como filtro no MDE Device Group. Todos devem ter o mesmo valor.

5. Etapa 3 — Resource Group

O Que Acontece

Cria um Resource Group dedicado para os recursos da automação MDE, com **8 tags corporativas** para rastreabilidade e governança.

📸 Saída do Terminal

```
Deploy-MDE-Automation.ps1 — Stage 3/14

[3/14] RESOURCE GROUP
=========================================================

  [OK] Resource Group criado: rg-mde-me-mngenvmcap186458-rafaelluizf-1
  [OK] Localização: eastus2
  [OK] 8 tags aplicadas
```

📸 No Portal — Resource Group com Tags

☁️ Azure Portal — Resource Groups

🏠 Home > Resource groups > rg-mde-me-mngenvmcap186458-rafaelluizf-1 > Tags

Tag Name	Value
Project	MDE-PolicyAutomation
Purpose	MDE Device Tag Management
ManagedBy	Deploy-MDE-Automation
Subscription	me-mngenvmcap186458-rafaelluizf-1
CreatedDate	2026-03-01
Version	1.0.4
AutomationType	PolicyDriven
SecurityLevel	Production

💡 **URL direta:** portal.azure.com → Resource Groups

6. Etapa 4 — Grupo Entra ID (Security Group)

O Que Acontece

Cria um Security Group no Entra ID (Azure AD) para agrupar os device objects das VMs da subscription. O Runbook (Etapa 10) populará este grupo automaticamente.

📸 Saída do Terminal

```
Deploy-MDE-Automation.ps1 — Stage 4/14

[4/14] ENTRA ID GROUP
==========================================================

  Verificando grupo existente...
  [OK] Grupo criado: MDE-ME-MngEnvMCAP186458-rafaelluizf-1
  [OK] Object ID: c6e6bfee-9e19-4495-97b0-0d5a6ca30e45
  [OK] Tipo: Security (Cloud)
```

📸 No Portal — Entra Admin Center

🔐 Microsoft Entra admin center — Groups

🏠 Groups > All groups > 🔍 "MDE-"

Display Name	Type	Source	Members
MDE-ME-MngEnvMCAP186458-rafaelluizf-1	Security	Cloud	8 devices

ℹ️ **Como funciona:** O grupo começa **vazio** após a criação. Ele será populado automaticamente pelo Runbook (Etapa 10) a cada hora, sincronizando VMs Azure/Arc com devices no Entra ID.

7. Etapa 5 — Automation Account

O Que Acontece

Provisiona um Azure Automation Account que servirá como o **"motor operacional"** — executando o Runbook PowerShell que sincroniza VMs com o Entra ID Group.

📸 Saída do Terminal

```
Deploy-MDE-Automation.ps1 — Stage 5/14

[5/14] AUTOMATION ACCOUNT
==========================================================

  Criando Automation Account...
  [OK] Automation Account: aa-mde-me-mngenvmcap186458-rafaelluizf-1
  [OK] SKU: Basic
  [OK] Localização: eastus2
```

📸 No Portal — Automation Account

☁️ Azure Portal — Automation Accounts

🏠 Automation Accounts > aa-mde-me-mngenvmcap186458-rafaelluizf-1 > Overview

Property	Value
Name	aa-mde-me-mngenvmcap186458-rafaelluizf-1
Resource group	rg-mde-me-mngenvmcap186458-rafaelluizf-1
Location	East US 2
Subscription	ME-MngEnvMCAP186458-rafaelluizf-1
Identity	✅ System assigned

8. Etapa 6 — Managed Identity (Zero Trust)

O Que Acontece

Ativa a **System Assigned Managed Identity** no Automation Account. Esta identidade é o **pilar Zero Trust** — permite que o Runbook acesse Azure e Graph API **sem armazenar credenciais**.

📸 Saída do Terminal

Deploy-MDE-Automation.ps1 — Stage 6/14

```
[6/14] MANAGED IDENTITY
============================================================

  [OK] Managed Identity ativada (System Assigned)
  [OK] Principal ID: df4baebf-4b2f-453d-b4f4-e4708db2aaed
  [OK] Tenant ID: 9217e954-8efe-4920-ae1b-700c507c9faa
```

📸 No Portal — Managed Identity

☁️ Azure Portal — Automation Account → Identity

🏠 Automation Accounts > aa-mde-... > Identity > System assigned

Setting	Value
Status	✅ On
Object (principal) ID	df4baebf-4b2f-453d-b4f4-e4708db2aaed

🔐 **Zero Trust:** Managed Identity elimina a necessidade de armazenar **senhas, chaves de API ou certificados**. A autenticação é gerenciada pelo Azure AD automaticamente.

9. Etapa 7 — RBAC Reader

O Que Acontece

Atribui a role **Reader** à Managed Identity no nível da **subscription**. Isso permite que o Runbook liste todas as VMs e Azure Arc machines.

📸 Saída do Terminal

```
Deploy-MDE-Automation.ps1 — Stage 7/14

[7/14] RBAC READER
============================================================

  [OK] Role 'Reader' atribuida à Managed Identity
  [OK] Scope: /subscriptions/fbb41bf3-dc95-4c71-8e14-396d3ed38b91
```

📸 No Portal — IAM Role Assignment

☁️ Azure Portal — Subscription → Access Control (IAM)

🏠 Subscriptions > ME-MngEnv... > Access control (IAM) > Role assignments

Name	Type	Role	Scope
aa-mde-me-mngenvmcap186458-rafaelluizf-1	Managed Identity	Reader	Subscription

ℹ️ **Princípio Least Privilege:** Reader é a **menor permissão possível** para listar recursos. Não permite criar, modificar ou deletar nenhum recurso Azure.

10. Etapa 8 — Permissões Graph API

O Que Acontece

Concede à Managed Identity as permissões no **Microsoft Graph API** necessárias para gerenciar o grupo Entra ID:

- **Group.ReadWrite.All** — ler e adicionar membros ao grupo
- **Device.Read.All** — buscar devices (VMs) registrados no Entra ID

📸 Saída do Terminal

```
Deploy-MDE-Automation.ps1 — Stage 8/14

[8/14] GRAPH API PERMISSIONS
==========================================================

  Buscando Service Principal do Microsoft Graph...
  [OK] Graph SP encontrado

  Concedendo Group.ReadWrite.All...
  [OK] Group.ReadWrite.All concedida

  Concedendo Device.Read.All...
  [OK] Device.Read.All concedida

  [INFO] Requer consent de Global Administrator
```

📸 No Portal — Enterprise Applications → API Permissions

🔐 Entra Admin Center — Enterprise Applications → Permissions

🏠 Enterprise applications > aa-mde-... > Permissions

API	Permission	Type	Status
Microsoft Graph	**Group.ReadWrite.All**	Application	✅ Granted
Microsoft Graph	**Device.Read.All**	Application	✅ Granted

⚠️ **Admin Consent:** As permissões `Application` requerem consentimento de um **Global Administrator**. O script tenta conceder automaticamente, mas pode ser necessário consentir manualmente no portal.

💡 **URL direta:** entra.microsoft.com → Enterprise applications

11. Etapa 9 — Módulo Az.Accounts

O Que Acontece

Importa o módulo **Az.Accounts** da PowerShell Gallery para o Automation Account. Este módulo é necessário para que o Runbook se autentique via Managed Identity.

📸 Saída do Terminal

```
Deploy-MDE-Automation.ps1 — Stage 9/14

[9/14] MODULO AZ.ACCOUNTS
==========================================================

  Verificando módulo Az.Accounts no Automation Account...
  Importando da PowerShell Gallery...
  [OK] Az.Accounts importado com sucesso
  [OK] Versão: 3.0.5
  [OK] Status: Available
```

📸 No Portal — Automation Account → Modules

☁️ Azure Portal — Automation Account → Modules

🏠 Automation Accounts > aa-mde-... > Shared Resources > Modules

Module Name	Version	Status	Source
Az.Accounts	3.0.5	✅ Available	PowerShell Gallery

12. Etapa 10 — Runbook PowerShell

O Que Acontece

Cria, importa e publica um **Runbook PowerShell** que executa a lógica de sincronização:

1. Autentica via Managed Identity (`Connect-AzAccount -Identity`)
2. Lista todas as VMs + Azure Arc machines na subscription
3. Para cada VM, busca o device correspondente no Entra ID (por nome)
4. Adiciona os devices encontrados ao Security Group

📸 Saída do Terminal

```
Deploy-MDE-Automation.ps1 — Stage 10/14

[10/14] RUNBOOK
==========================================================

  Criando Runbook inline (PowerShell 5.1)...
  [OK] Runbook criado: rb-mde-sync-me-mngenvmcap186458-rafaelluizf-1
  [OK] Tipo: PowerShell
  [OK] Status: Published
```

📸 No Portal — Runbook

☁️ Azure Portal — Automation Account → Runbooks

🏠 Automation Accounts > aa-mde-... > Process Automation > Runbooks

Name	Type	Status	Last Modified
rb-mde-sync-me-mngenvmcap186458-rafaelluizf-1	PowerShell	✅ Published	2026-03-01

Lógica do Runbook (simplificada):

```
Runbook — Fluxo Lógico

1. Connect-AzAccount -Identity        # Auth via Managed Identity
2. Get-AzVM                               # Lista VMs Azure
3. Get-AzConnectedMachine                 # Lista Azure Arc machines
4. GET graph.../devices?$filter=...      # Busca device Entra ID por nome
5. POST graph.../groups/{id}/members/$ref # Adiciona device ao grupo
6. Loop para cada VM/Arc → repete 4-5
```

13. Etapa 11 — Schedule e Job Schedule

O Que Acontece

Cria um **Schedule** com frequência horária e o vincula ao Runbook via **Job Schedule**. A partir deste momento, a sincronização VM↔Entra ID ocorre **automaticamente a cada hora**.

📸 Saída do Terminal

```
Deploy-MDE-Automation.ps1 — Stage 11/14

[11/14] SCHEDULE + JOB SCHEDULE
==========================================================

  [OK] Schedule criado: sch-mde-me-mngenvmcap186458-rafaelluizf-1
  [OK] Frequência: Hourly (1 hora)
  [OK] Start Time: 2026-03-01T12:00:00Z
  [OK] Timezone: UTC
  [OK] Job Schedule vinculado ao Runbook
```

📸 No Portal — Schedule

☁️ Azure Portal — Automation Account → Schedules

🏠 Automation Accounts > aa-mde-... > Shared Resources > Schedules

Name	Frequency	Status	Next Run (UTC)
sch-mde-me-mngenvmcap186458-rafaelluizf-1	Every 1 hour	✅ Enabled	2026-03-01 13:00

📸 No Portal — Jobs (Histórico de Execuções)

☁️ Azure Portal — Automation Account → Jobs

🏠 Automation Accounts > aa-mde-... > Process Automation > Jobs

Runbook	Status	Start Time	Duration
rb-mde-sync-...	✅ Completed	2026-03-01 12:00	45s
rb-mde-sync-...	✅ Completed	2026-03-01 11:00	42s
rb-mde-sync-...	✅ Completed	2026-03-01 10:00	48s

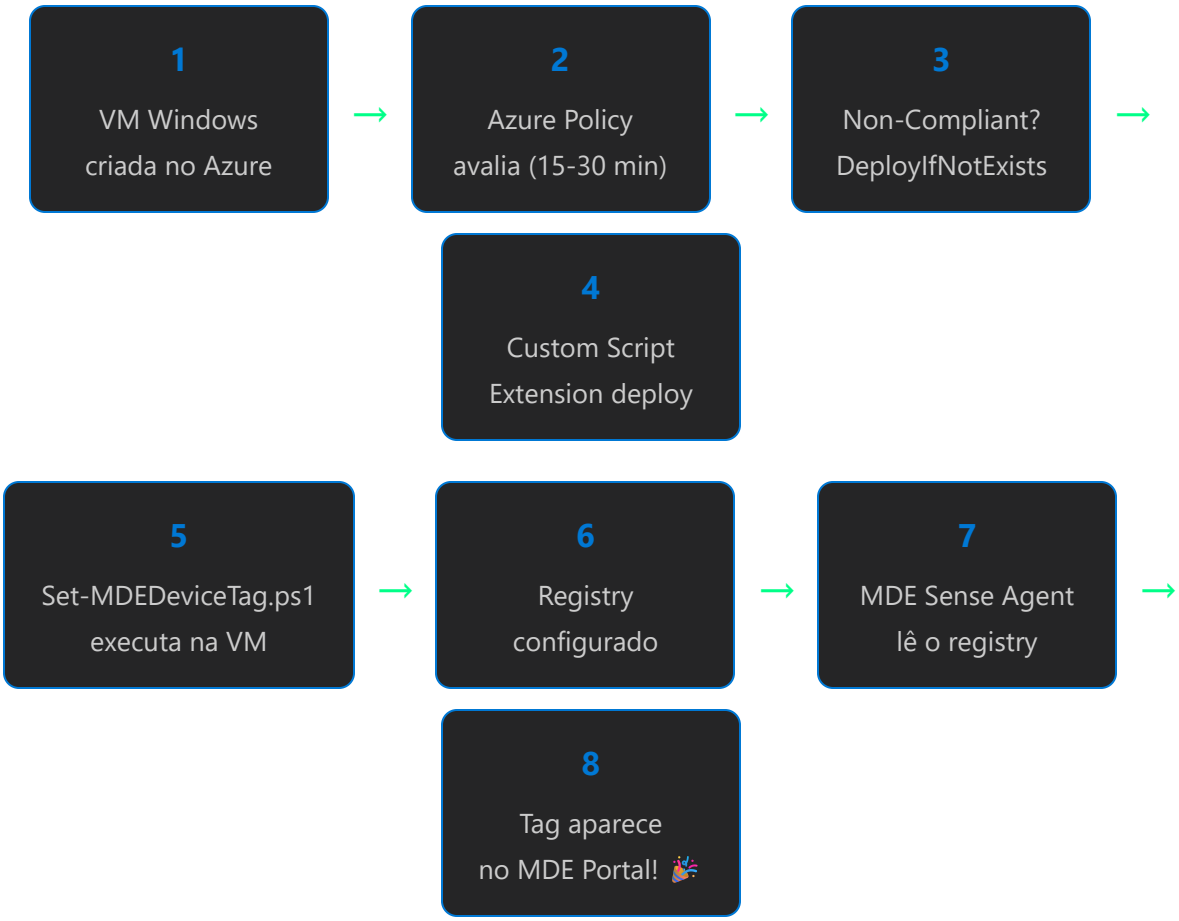
💡 **Dica:** Clique em qualquer job para ver o **Output** detalhado: quantas VMs foram encontradas, quantos devices adicionados ao grupo, erros (se houver).

14. Etapa 12 — Azure Policy (DeployIfNotExists)

O Que Acontece

Esta é a **etapa mais importante** para a automação de tags. Cria uma Azure Policy do tipo `DeployIfNotExists` que automaticamente configura o registry de cada VM Windows com o tag da subscription.

📸 Fluxo da Azure Policy



📸 No Portal — Policy Definitions

☁️ Azure Portal — Policy → Definitions

🏠 Policy > Definitions > 🔍 "mde-device-tag"

Name	Type	Mode	Effect
Deploy MDE Device Tag to Windows Virtual Machines mde-device-tag-windows-vms	Custom	Indexed	DeployIfNotExists

📋 Policy Rule (resumo):

policy-definition.json

```
IF:
  type == "Microsoft.Compute/virtualMachines"
  AND osType == "Windows"

THEN (DeployIfNotExists):
  → Cria sub-resource: CustomScriptExtension
  → Nome: MDEDeviceTagExtension
  → Script: Set-MDEDeviceTag.ps1
  → Args: -TagValue "mde-policy-{sub}"

Parameters:
  tagValue:  string (required) — valor do tag MDE
  scriptUri: string (required) — URL do script no GitHub/Storage
  effect:    string (default: DeployIfNotExists)
```

💡 **URL direta:** portal.azure.com → Policy → Definitions

📸 No Portal — Policy Compliance

☁️ Azure Portal — Policy → Compliance

🏠 Policy > Compliance

Assignment Name	Compliance	Non-Compliant Resources
MDE Device Tag | SecurityLab mde-tag-securitylab	✅ Compliant	0

Resource Compliance:

Resource	Compliance State	Last Evaluated
lab-agencia4	✅ Compliant	2026-03-01
lab-mde-policy-tag-mng	✅ Compliant	2026-03-01

📸 No Portal — VM Extensions

☁️ Azure Portal — VM → Extensions + applications

🏠 Virtual machines > lab-agencia4 > Extensions + applications

Name	Publisher	Type	Status
MDEDeviceTagExtension	Microsoft.Compute	CustomScriptExtension	✅ Provisionir succeeded
AzurePolicyForWindows	Microsoft.GuestConfiguration	ConfigurationforWindows	✅ Provisionir succeeded

ℹ️ **Nota:** A extensão `MDEDeviceTagExtension` foi criada **automaticamente** pela Azure Policy. Não é necessário instalar manualmente.

📸 Verificação do Registry na VM

PowerShell — Verificação Remota via az vm run-command

```
PS C:\> az vm run-command invoke -g SecurityLab -n lab-agencia4 `
    --command-id RunPowerShellScript `
    --scripts "Get-ItemProperty 'HKLM:\SOFTWARE\Policies\Microsoft\Windows Advanced
Threat Protection\DeviceTagging'"

SENSE: Running
TAG: mde-policy-me-mngenvmcap186458-rafaelluizf-1
```

```
📸 Registry Editor (dentro da VM):

📁 Computer
  📁 HKEY_LOCAL_MACHINE
    📁 SOFTWARE
      📁 Policies
        📁 Microsoft
          📁 Windows Advanced Threat Protection
            📁 DeviceTagging

              Name    Type       Data
              Group   REG_SZ     mde-policy-me-mngenvmcap186458-rafaelluizf-1
```

15. Etapa 13 — MDE Device Groups

O Que Acontece

O script gera um arquivo HTML com instruções detalhadas para criar o MDE Device Group no portal. Esta é a **única etapa manual** — a API do MDE não suporta criação programática de Device Groups.

📸 No Portal MDE — Criação do Device Group

🛡️ Microsoft Defender XDR — Settings → Endpoints → Device groups

⚙️ Settings > Endpoints > Device groups > + Add device group

Step 1: General

Campo	Valor a Preencher
Device group name	`mde-policy-me-mngenvmcap186458-rafaelluizf-1`
Description	Managed device group for subscription ME-MngEnvMCAP186458-rafaelluizf-1
Automation level	🔵 Full - remediate threats automatically

Step 2: Devices (Filtro)

Campo	Valor
Filter by	Tag
Operator	Equals
Value	`mde-policy-me-mngenvmcap186458-rafaelluizf-1`

Preview

Device Name	OS	Tag
LAB-AGENCIA4	Windows 10	🏷️ mde-policy-me-mngenvmcap186458-rafaelluizf-1
LAB-MDE-POLICY-TAG-MNG	Windows 10	🏷️ mde-policy-me-mngenvmcap186458-rafaelluizf-1

⚠️ **IMPORTANTE:** O nome do Device Group (`mde-policy-{sub}`) deve corresponder **exatamente** ao valor da tag configurada no registry. Assim, todos os devices com essa tag serão automaticamente filtrados para o grupo.

💡 **URL direta:** security.microsoft.com → Settings → Endpoints → Device groups

📸 No Portal MDE — Device Inventory com Tags

🛡️ Microsoft Defender XDR — Assets → Devices

🏠 Assets > Devices > 🔍 Filter: Tag contains "mde-policy"

Device Name	OS	Tags	Device Group	MDE Onboarding
LAB-AGENCIA4	Windows 10	🏷️ mde-policy-me-mngenvmcap186458-rafaelluizf-1	mde-policy-me-mngenvmcap186458-rafaelluizf-1	✅ Active
LAB-MDE-POLICY-TAG-MNG	Windows 10	🏷️ mde-policy-me-mngenvmcap186458-rafaelluizf-1	mde-policy-me-mngenvmcap186458-rafaelluizf-1	✅ Active

Detalhe do Device — Tags Section:

Tags:

🏷️ `mde-policy-me-mngenvmcap186458-rafaelluizf-1`

ℹ️ Tag definida via registro Windows (Azure Policy DeployIfNotExists)

💡 **URL direta:** security.microsoft.com → Assets → Devices

16. Etapa 14 — MDE Machine Tags via API

O Que Acontece

Configura uma **App Registration** no Entra ID com permissões `WindowsDefenderATP` para gerenciar Machine Tags via API. Isto permite operações avançadas como adicionar/remover tags programaticamente.

📸 Saída do Terminal

```
Deploy-MDE-Automation.ps1 — Stage 14/14

[14/14] MDE MACHINE TAGS VIA API
============================================================

  Criando App Registration...
  [OK] App: MDE-TagManager-me-mngenvmcap186458
  [OK] App ID: xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx

  Adicionando API permissions: WindowsDefenderATP...
  [OK] Machine.ReadWrite.All concedida
  [OK] Machine.Read.All concedida

  Criando client secret...
  [OK] Secret criado (expira em 1 ano)
  [SALVE] Client Secret: xxxxxxxx~xxxxxxxxxxxxxxxxxxxxxxxxxxx

============================================================
  ✅ DEPLOYMENT COMPLETO - 14/14 STAGES FINALIZADOS
============================================================
```

📸 No Portal — App Registration

🔐 Entra Admin Center — App registrations

🏠 App registrations > MDE-TagManager-me-mngenvmcap186458 > API permissions

API	Permission	Type	Status
WindowsDefenderATP	**Machine.ReadWrite.All**	Application	✅ Granted
WindowsDefenderATP	**Machine.Read.All**	Application	✅ Granted

📡 Como Usar a API MDE

PowerShell — Exemplo de uso da MDE API

```
# 1. Obter token OAuth2
PS C:\> $body = @{
    client_id     = $appId
    client_secret = $secret
    scope         = "https://api.securitycenter.microsoft.com/.default"
    grant_type    = "client_credentials"
}
PS C:\> $token = (Invoke-RestMethod -Method POST `
    -Uri "https://login.microsoftonline.com/$tenantId/oauth2/v2.0/token" `
    -Body $body).access_token

# 2. Listar machines com tags
PS C:\> $headers = @{ Authorization = "Bearer $token" }
PS C:\> Invoke-RestMethod `
    -Uri "https://api.securitycenter.microsoft.com/api/machines" `
    -Headers $headers | Select machineTags

# 3. Adicionar tag a um device
PS C:\> $body = @{ Value = "mde-policy-production"; Action = "Add" } | ConvertTo-Json
PS C:\> Invoke-RestMethod -Method POST `
    -Uri "https://api.securitycenter.microsoft.com/api/machines/{id}/tags" `
    -Headers $headers -Body $body -ContentType "application/json"

[OK] Tag adicionada com sucesso
```

📸 Detalhe: Entra ID — Grupo e Membros (Referência Visual)

🔐 Microsoft Entra admin center — Groups

🏠 Groups > All groups > MDE-ME-MngEnvMCAP186458-rafaelluizf-1 > Members

Group Details

Property	Value
Display Name	**MDE-ME-MngEnvMCAP186458-rafaelluizf-1**
Type	Security
Source	Cloud
Object ID	c6e6bfee-9e19-4495-97b0-0d5a6ca30e45

Members (8 devices)

Display Name	Object Type	OS
LAB-AGENCIA4	Device	Windows
SecLab-DC	Device	Windows
tagDEV	Device	Windows
sap-sentinel-agent	Device	Windows
vm-exposed-lab	Device	Windows
Sentinel4sa-Wi3	Device	Windows
sid-j2e	Device	Windows
ip-172-31-29-155...	Device	Windows

ℹ️ **Preenchido automaticamente:** O Runbook do Automation Account sincroniza VMs Azure com este grupo **a cada hora**. Não é necessário adicionar devices manualmente.

💡 **URL direta:** entra.microsoft.com → Groups → All groups

17. Validação Completa

✅ Checklist de Verificação

- ✅ **1.** Resource Group existe com 8 tags — *Portal → Resource Groups → rg-mde-{sub} → Tags*
- ✅ **2.** Entra ID Group existe — *Entra Admin Center → Groups → grp-mde-{sub}*
- ✅ **3.** Entra ID Group tem membros (devices) — *Group → Members*
- ✅ **4.** Automation Account com Managed Identity — *Portal → Automation Accounts → aa-mde-{sub} → Identity*
- ✅ **5.** Runbook Published — *Automation Accounts → Runbooks*
- ✅ **6.** Schedule Enabled (hourly) — *Automation Accounts → Schedules*
- ✅ **7.** Azure Policy Assigned — *Portal → Policy → Assignments*
- ✅ **8.** VMs com registry tag configurado — *az vm run-command invoke*
- ✅ **9.** MDE Portal mostra tag nos devices — *security.microsoft.com → Devices*
- ✅ **10.** MDE Device Group criado — *security.microsoft.com → Device groups*

📸 Comandos de Validação

PowerShell — Validação Completa

```
# 1. Verificar Policy Definition
PS C:\> az policy definition show --name "mde-device-tag-windows-vms" --query "
{name:name,mode:mode}" -o table
Name                        Mode
--------------------------  -------
mde-device-tag-windows-vms  Indexed

# 2. Verificar Tag no Registry da VM (remoto)
PS C:\> az vm run-command invoke -g SecurityLab -n lab-agencia4 `
    --command-id RunPowerShellScript `
    --scripts "Write-Output ('SENSE: ' + (Get-Service Sense).Status); `
               Write-Output ('TAG: ' + (Get-ItemProperty 'HKLM:\...DeviceTagging' -
Name Group).Group)"

SENSE: Running
TAG: mde-policy-me-mngenvmcap186458-rafaelluizf-1

# 3. Verificar membros do Entra ID Group
PS C:\> $result = Invoke-RestMethod -Uri
"https://graph.microsoft.com/v1.0/groups/{id}/members" -Headers $headers
Total: 8 devices
LAB-AGENCIA4, SecLab-DC, tagDEV, sap-sentinel-agent, vm-exposed-lab, ...

# 4. Verificar Policy Compliance
PS C:\> az policy state summarize --scope
"/subscriptions/.../resourceGroups/SecurityLab"
Compliant: 2 | Non-Compliant: 0
```

18. Troubleshooting

❌ Problema: Tag não aparece no MDE Portal

1. **Verificar MDE Sense Agent:** `Get-Service -Name Sense` → deve estar `Running` com StartType `Automatic`
2. **Verificar registry:** `Get-ItemProperty "HKLM:\...\DeviceTagging"` → `Group` deve ter valor correto
3. **Aguardar sincronização:** MDE sincroniza tags a cada 15-30 minutos. Primeira vez pode levar até 4 horas
4. **Verificar conectividade:** VM precisa acessar `*.securitycenter.windows.com`

⚠️ Problema: Policy mostra "Non-Compliant"

1. **VM é Windows?** Policy só se aplica a VMs Windows (osType == Windows)
2. **Managed Identity da Assignment:** Precisa de role `Virtual Machine Contributor`
3. **Script acessível?** Testar URL: `Invoke-WebRequest -Uri $ScriptUri`
4. **Forçar avaliação:** `az policy state trigger-scan --resource-group "rg-mde-*"`

🔑 Problema: Runbook falha com erro de permissão

1. **Managed Identity ativada?** Automation Account → Identity → System assigned → On
2. **RBAC Reader?** Subscription → IAM → MI deve ter Reader
3. **Graph API permissions?** Enterprise Apps → API permissions → Group.ReadWrite.All + Device.Read.All
4. **Módulo Az.Accounts?** Automation Account → Modules → Status = Available

19. Referências e URLs do Portal

Portal	URL	O Que Verificar
Azure Policy - Definitions	portal.azure.com/...Definitions	Definição mde-device-tag
Azure Policy - Compliance	portal.azure.com/...Compliance	Status compliance
Resource Groups	portal.azure.com/...ResourceGroups	RG rg-mde-{sub}
Automation Accounts	portal.azure.com/...AutomationAccounts	Runbooks, Identity
Entra ID Groups	entra.microsoft.com/...AllGroups	Grupo e membros
MDE Devices	security.microsoft.com/machines	Tags nos devices
MDE Device Groups	security.microsoft.com/.../device_groups	Criar Device Groups
MDE Tags Docs	learn.microsoft.com/.../machine-tags	Documentação oficial

Documentação Oficial Microsoft

- Azure Policy Overview
- DeployIfNotExists Effect
- Managed Identities for Azure Resources
- MDE Machine Tags
- MDE Device Groups
- Azure Automation Runbooks
- Custom Script Extension for Windows

"From manual chaos to automated governance — one policy at a time."

Autor: Rafael França — Customer Success Architect, Cyber Security @ Microsoft

GitHub: github.com/rfranca777/MDE-PolicyAutomation

Comunidade: ODefender Community